

AGENTS

Helix Constitution — Universal AGENTS.md

This is the **base AGENTS.md** for any CLI agent (Codex, Cursor, Aider, Continue, Gemini CLI, future LLMs) working on a project that includes the Helix Constitution submodule. Project-level AGENTS.md may extend or tighten rules but **MUST NOT** weaken them.

Last revision: 2026-05-14

How inheritance works

Because not every CLI agent honours the Markdown @import syntax, each consuming project's root AGENTS.md **MUST** do one of:

1. **Reference + restate critical rules** (safest — works for every agent):

```
# <Project> – AGENTS.md
```

```
> Base agent rules: `constitution/AGENTS.md` – READ IT FIRST.  
> The base file is authoritative for any topic not covered  
   here.
```

```
## Critical base rules restated (for agents that don't follow  
   links)
```

- Never commit secrets.
- All commits go through the project's commit wrapper.
- Anti-bluff covenant binds every test (Constitution §11.4).

```
## Project-specific
```

```
- ...
```

2. **Generate at build time** — keep one source-of-truth in this submodule, concatenate with a project-specific delta file. A `compose-agents.sh` script in this submodule's `Upstreams/` (or `scripts/` if added) would output `constitution/AGENTS.md` followed by `---` followed by `AGENTS.project.md`.

Identity & posture

You are working inside a project that has opted into the Helix Constitution. The Constitution is the source of truth for engineering discipline (test coverage, anti-bluff covenant, data safety, host safety, credentials handling, documentation discipline). Whenever this `AGENTS.md` mentions a `$X.Y` reference, it points to `constitution/Constitution.md`.

The Constitution applies recursively to every submodule of every project that includes it — you **MUST** treat the Constitution's rules as in-force regardless of how deep into the submodule tree you are working.

Top-level invariants for every agent

1. **Never assume; verify by reading files.** Especially before any destructive action.
2. **Never bluff.** Do not invent file paths, command outputs, test results, or library APIs you have not verified. If you are unsure, say so explicitly and verify.
3. **No guessing language** (§11.4.6). `likely`, `probably`, `maybe`, `might`, `appears`, `seems` are forbidden when reporting causes. Use captured evidence or mark explicitly `UNCONFIRMED`.
4. **Test coverage for every change** (§1) — pre-build gate, post-build gate, runtime test, AND a meta-test mutation that proves the gate catches regressions.
5. **Commit through the project's official wrapper.** Direct `git add / git commit / git push` on the main repo is forbidden in normal workflow.
6. **Never skip hooks** (`--no-verify`, `--no-gpg-sign`).
7. **Never force-push.** Force-push requires explicit per-session human authorization AND a green §9.1.5 post-op gate.
8. **Hardlinked backup before any destructive op** (§9). Zero excuse.
9. **Host safety** (§12) — abort if pre-flight check fails; wrap heavy work in bounded execution scopes; never exceed 60% of host RAM.
10. **Credentials NEVER tracked** (§11.4.10) — `.env` patterns git-ignored; runtime-load only; per-service file separation. **Pre-store leak audit** (§11.4.10.A, 2026-05-17) — before storing operator-provided credentials in any gitignored config, grep

the entire tracked tree AND git history for the literal value(s); surface any findings to operator before storing.

11. **CONTINUATION.md kept in sync** (§12.10) — every non-trivial state change updates the continuation document in the same commit.

Critical base rules restated (for agents that don't honour @imports)

Anti-bluff covenant — END-USER QUALITY GUARANTEE (§11.4)

Forensic anchor — verbatim user mandate (2026-04-28):

"We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This **MUST NOT** be the case and execution of tests and Challenges **MUST** guarantee the quality, the completion and full usability by end users of the product!"

The bar for shipping is **not** "tests pass" but "**users can use the feature.**" Every PASS **MUST** carry positive evidence captured during execution that the feature works for the end user. Metadata-only PASS, configuration-only PASS, absence-of-error PASS, and grep-based PASS without runtime evidence are critical defects.

Mutation-paired gates (§1.1)

Every new gate **MUST** be paired with a mutation entry in the project's meta-test harness that mutates the asserted condition and asserts the gate now FAILs. A gate without a paired mutation is a bluff gate and is forbidden.

Recorded-evidence requirement (§11.4.2)

Every PASS for a user-visible feature **MUST** be cross-checked against a captured recording + action timeline. A PASS that lacks at least one matched timeline event in the analyzer findings is a §11.4 PASS-bluff.

Test-interrupt-on-discovery (§11.4.4)

The moment any defect is re-discovered, re-produced, or newly identified during a test cycle, the cycle MUST stop. Then: systematic debugging → fix at root cause → four-layer test coverage (pre-build / post-build / runtime / meta-test paired mutation) → full re-build → re-deploy on every target → full retest from beginning.

No-guessing mandate (§11.4.6)

Forensic anchor — verbatim user mandate (2026-05-08):

"'LIKELY' is guessing, we MUST NOT have guessing, since it can be or may not be! No bluffing and uncertainty is allowed at any cost! We MUST always know exactly precisely what is happening exactly, in any context, under any conditions, everywhere!"

Forbidden vocabulary when reporting causes: likely, probably, maybe, might, possibly, presumably, seems, appears to, guess, seemingly, apparently, perhaps, supposedly, conjectured, and synonyms.

Item-type tracking (§11.4.16)

Every active item in the project Issues file MUST carry a ****Type:**** line within eight non-blank lines of its heading. Three-value CLOSED vocabulary: Bug (product defect / regression / user-visible broken behaviour), Feature (new capability not previously offered to end users), Task (internal workstream — refactor, doc, infra, gate, audit; the lowest-stakes default when ambiguous). The Issues_Summary file carries the Type column for every active item. All three Issues / Issues_Summary / Fixed file types kept in sync (Markdown + HTML + PDF). Pre-build gates CM-ITEM-TYPE-TRACKING + CM-COVENANT-114-16-PROPAGATION enforce the mandate. No escape hatch.

Universal-vs-project classification (§11.4.17, User mandate 2026-05-14)

Before adding ANY new rule / mandatory constraint / covenant clause / gate / "MUST"-bearing statement, classify it: **universal** (any project → constitution submodule) or **project-specific** (specific hardware / vendor / package / region → project / submodule layer). Commit message MUST carry Classification: line + one-sentence rationale. When uncertain, default to project-specific. Gate CM-UNIVERSAL-VS-PROJECT-CLASSIFICATION audits new rule commits. Paired mutation enforces the gate is not a bluff.

Subagent-driven-by-default (§11.4.20, User mandate 2026-05-14)

When the runtime supports subagent delegation (Agent tool / task runner / sub-session), the primary agent **MUST** default to sub-agent delegation for multi-step scope (≥ 3 phases), parallelisable independent subtasks, long-running diagnostic loops, OR specialised domain workflows. Foreground-only is reserved for single-file edits, mid-execution operator clarification, critical-state sequencing (commits / pushes / tags), or tasks under ~ 30 s. Discipline: tight scope (4-6 tasks), checkpoint commits per task, anti-stall protection in prompts, anti-bluff verification of subagent claims via repo state. Parallel subagents **MUST** partition non-overlapping files; parent `commit_all.sh --auto-cascade` bundles via `git add -A`. Gate CM-SUBAGENT-DELEGATION-AUDIT (per consuming project) flags foreground multi-step work as a violation. Paired mutation enforces the gate is not a bluff.

Script documentation mandate (§11.4.18, User mandate 2026-05-14)

Every Bash / shell / POSIX-sh script under `scripts/` / `bin/` / `tests/` / library dirs / CI hooks (depth-N recursive) **MUST** carry (1) an in-source documentation block at the top (Purpose / Usage / Inputs / Outputs / Side-effects / Dependencies / Cross-references) AND (2) an external user guide under `docs/scripts/<script-name>.md` (Overview / Prerequisites / Usage examples / Edge cases / Internal behaviour / Related scripts / Last verified date). When a script is modified, **BOTH** the in-source block AND the external user guide **MUST** be updated in the SAME commit. **No documentation ever out of sync with its codebase.** Gate CM-SCRIPT-DOCS-SYNC enforces. Paired mutation strips the invariant.

Fixed-document column-alignment (§11.4.19, User mandate 2026-05-14)

The closed-archive tracker (`Fixed.md`) **MUST** mirror the open-work tracker (`Issues.md` + `Issues_Summary.md`) along the same lifecycle axes — at minimum **Status** and **Type**. Concretely: (1) every `###` / `####` heading in `Fixed.md` carries ****Status:**** (from closed-set {Fixed ($\rightarrow$ Fixed.md) | Fixed – pending device verification | Fixed – RECLASSIFIED}) and ****Type:**** ({Bug | Feature | Task}) within 8 non-blank lines of the heading; (2) `Fixed_Summary.md` companion exists with column structure `# | Level | Status | Type | One-line description` matching `Issues_Summary.md` exactly; (3) all three formats (`.md`, `.html`, `.pdf`) for BOTH `Fixed` and `Fixed_Summary` stay in sync via the same single-shot wrapper that handles `Issues` + `Issues_Summary`; (4) closure is atomic — resolved items migrate from `Issues.md` to `Fixed.md` in the same commit. Pre-build gate CM-FIXED-COLUMN-ALIGNMENT (5+ invariants: file-exists, header-has-

Status+Type, mtime $\geq$ Fixed.md, generator script present, sync wrapper invokes it, HTML+PDF exports present). Paired mutation strips Status column from Fixed_Summary header $\rightarrow$ gate FAILs. Classification: universal (per §11.4.17).

Operator-blocked status + self-resolution exhaustion (§11.4.21, User mandate 2026-05-14)

Operator-blocked is the §11.4.15 Status closed-set's 7th value alongside

{Queued | In progress | Ready for testing | In testing | Reopened | Fixed ($\rightarrow$ Fixed.md)}. **Last-resort classification** — agent MUST first exhaust applicable self-resolution paths: (a) CLI / ADB / SSH / API access already available, (b) subagent delegation per §11.4.20, (c) existing repo tooling, (d) captured fallback (synthetic event / asset substitution / §11.4.3 SKIP), (e) external research per §11.4.8. Every Operator-blocked item MUST carry ****Operator-Block-Details:**** within 8 non-blank lines of its heading stating WHAT (action) / WHY (each exhausted alternative) / UNBLOCK CONDITION (observable signal) / WHO (contact or doc pointer).

Issues_Summary.md lists it as a sortable Status value. Items re-evaluated every Nth tag cycle ($\geq$ 3rd recommended). Fake Operator-blocked without exhaustion audit = §11.4 covenant violation at planning layer (PASS-bluff equivalent). Gates CM-ITEM-OPERATOR-BLOCKED-DETAILS + CM-OPERATOR-BLOCKED-SELF-RESOLUTION-AUDIT (every NEW Operator-blocked commit carries an "Attempted: a — ...; b — ...; c — ..." trail). Classification: universal (per §11.4.17). No escape hatch.

Document-sync commit discipline (§11.4.22, User mandate 2026-05-14)

Every project tracking work items through an Issues / Fixed lifecycle MUST provide a **lightweight commit path** distinct from the full-repo commit wrapper. Stages, commits, pushes ONLY the status-tracking doc set: Issues + Issues_Summary + Fixed + Fixed_Summary + CONTINUATION + their HTML + PDF exports + auto-generated audit artifact. Wrapper MUST: (a) auto-invoke the project's export-regeneration pipeline first so Markdown + HTML + PDF stay in sync; (b) stage ONLY the explicit doc-set list — NEVER `git add -A`; (c) use a separate flock disjoint from the full-tree wrapper; (d) push to every parent-repo remote; (e) exit 3 on nothing-to-commit (informational). MUST be invocable standalone OR via a delegation flag on the full-tree wrapper (e.g. `commit_all.sh --docs-only`). Inherits §9 preflight. Gate CM-COMMIT-DOCS-EXISTS + paired mutation. Composes with §11.4.12 / §11.4.15 / §11.4.18 / §12.10. Classification: universal (per §11.4.17). No escape hatch — doc-status drift is a §11.4 PASS-bluff at the documentation layer.

Build-resource stats tracking (§11.4.24, User mandate 2026-05-14)

Every project under this Constitution with a build exceeding 1 minute wall-clock MUST run a host-side resource sampler for every build — samples `/proc/meminfo` + `/proc/loadavg` + `/proc/stat` + `/proc/diskstats` at a fixed interval (recommended 5 s); writes TSV. On stop, computes **min** / **max** / **mean** / **p95** per metric; appends one TSV row per build to a registry; regenerates a Markdown report (`Stats.md`) whose top surfaces **ever-values** (min / max / mean across all tracked builds) and whose body lists per-build entries most-recent-first with SUCCESS / FAIL / UNKNOWN + reason. `Stats.md` exported to `Stats.html` + `Stats.pdf` through the project's normal export pipeline per §11.4.12. Triple committed via the §11.4.22 lightweight doc-sync wrapper. Sampler MUST stay under 50 MB RSS / 5% CPU. Stop hook called from both success AND failure paths of the build wrapper. Gate CM-BUILD-RESOURCE-STATS-TRACKER

- paired mutation. Classification: mixed (per §11.4.17). Composes with §11.4.12 / §11.4.18 / §11.4.22 / §12.6 / §12.7 / §12.9. No escape hatch — build-resource debugging without time-series data is the bluff this anchor forbids.

Full-Automation-Coverage (§11.4.25, User mandate 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

"Make sure that every feature, every functionality, every flow, every use case, every edge case, every service or application, on every platform we support is covered with full automation tests which will confirm anti-bluff policy and provide the proof of fully working capabilities, working implementation as expected, no issues, no bugs, fully documented, tests covered!"

No feature / functionality / flow / use case / edge case / service / application on any supported platform may be considered deliverable until automation tests prove six invariants: (1) anti-bluff posture with captured runtime evidence (§7.1 + §11.4); (2) proof of working capability end-to-end on target topology (§11.4.3, no mocks); (3) implementation matches documented promise; (4) no open issues/bugs surfaced (cross-checked vs §11.4.15 / §11.4.16); (5) full documentation (user manual + §11.4.18 for scripts) kept in sync via §11.4.12; (6) four-layer test floor per §1 (pre-build + post-build + runtime + paired mutation). Consuming projects publish a coverage ledger (feature × platform × invariant × status) regen-

erated at release-gate sweep. Classification: universal (§11.4.17). Severity-equivalent to a §11.4 PASS-bluff at the release-gate layer. No escape hatch. See Constitution §11.4.25 for the full mandate.

Constitution-Submodule Update Workflow (§11.4.26, User mandate 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

"Every time we add something into our root (constitution Submodule) Constitution, CLAUDE.MD and AGENTS.MD we MUST FIRST fetch and pull all new changes / work from constitution Submodule first! All changes we apply MUST BE committed and pushed to all constitution Submodule upstreams! In case of conflict, IT MUST BE carefully resolved! Nothing can be broken, made faulty, corrupted or unusable! After merging full validation and verification MUST BE done!"

Before any modification to constitution/
{Constitution,CLAUDE,AGENTS}.md, execute in order: (1) fetch + pull (--ff-only or --rebase; never strategy=ours / unrelated-histories without authorization) so the submodule is at upstream tip before editing; (2) apply the change with §11.4.17 classification + verbatim mandate quote; (3) validate (meta_test_inheritance.sh + no merge-conflict markers + cross-file consistency); (4) commit (governance files only, no git add -A)

- push to EVERY configured upstream remote (§2.1 violation if not); (5) careful conflict resolution preserving union of governance content — force-push forbidden (§9.2); (6) post-merge validation: git submodule update --remote --init + re-run cascade verifier (CONST-047) confirming the new clause reaches every owned submodule; (7) bump consuming project's .gitmodules pointer to new HEAD in the SAME commit as cascade work — out-of-sync pointer = §11.4.26 violation. Classification: universal (§11.4.17). Severity-equivalent to force-push without §9.2 authorization. No escape hatch. See Constitution §11.4.26 for the full pipeline (operational scope, cross-cutting reach).

Submodule-dependency-manifest (§11.4.31, User mandate 2026-05-15)

Every owned-by-us submodule MUST ship helix-deps.yaml listing its own-org dependencies: {name, ssh_url, ref, why, layout: flat|grouped}. Tooling incorporate-submodule <ssh-url> adds the submodule at the parent project's canonical path (CONST-051(C)) + reads its helix-deps.yaml + recurses + aborts on conflicting refs + emits <root>/.helix-manifest.yaml audit record. Anti-bluff: each manifest paired with a Challenge that bootstraps from scratch, asserts lay-

out matches manifest, runs submodule tests, captures wire evidence. Without the proof, the manifest is a §11.4.31 violation. This rule is the operational complement of §11.4.28 / CONST-051(C) — manifests are the bridge that lets consumers reconstruct the dependency graph at the parent root. Classification: universal (§11.4.17). See Constitution §11.4.31 for the full mandate.

Post-constitution-pull validation (§11.4.32, User mandate 2026-05-15)

Whenever a project's constitution submodule is fetched + pulled with any content change, run `scripts/verify-all-constitution-rules.sh` BEFORE the new HEAD is treated as canonical for other work. The sweep re-runs the governance-cascade verifier + every implementable rule gate (CONST-053 `.gitignore` audit, CONST-051(C) nested-own-org audit, CONST-052 case audit, CONST-050(A) mock-from-production audit, CONST-035 anti-bluff smoke). Failures populate Issues per §11.4.15 (Status: Reopened, Type: Bug); closure requires positive-evidence per §11.4. Pull-time invocation auto-triggered by `git submodule update --remote constitution`. Anti-bluff: sweep's own meta-test (paired mutation §1.1) plants a violation per gate, asserts sweep FAILs. This is the **enforcement engine** for every other rule — without it, new rules are decorative anchors rather than enforced gates. Classification: universal (§11.4.17). See Constitution §11.4.32 for the full mandate.

.gitignore + no-versioned-build-artifacts (§11.4.30, User mandate 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

"every project module, every Submodule, every service and application MUST HAVE proper `.gitignore` file! We MUST NOT git version build artifacts, cache files, tmp files, main `.env` file(s) or any files containing sensitive data, API keys or token! ... If any violation is detected it MUST be fixed before commit is executed!"

Every project module / submodule / service / application MUST ship a proper `.gitignore`. Forbidden-from-version-control: build artefacts (`bin/`, `build/`, `dist/`, `target/`, `*.exe`, `*.so`, `*.class`, `*.pyc`), cache files (`__pycache__`, `node_modules/`, `.gradle/`, `.terraform/`), temp files (`*.tmp`, `*.swp`, `.DS_Store`), sensitive data (`.env`, `.env.*` except `.env.example` placeholder, `*.pem`, `*.key`, `id_rsa*`, `.netrc`, `secrets/`, `api_keys.sh`), generated logs (`*.log`, `coverage.out`, `htmlcov/`), OS/IDE personal state (`.idea/`, `.history/`).

Anti-bluff invariant: ignore-line alone is not enough — no file matching forbidden patterns may be tracked. Pre-commit attention: every author inspects `git diff --staged + git status BE-`

FORE commit; violations abort the commit (un-stage, add to ignore, scrub if already-tracked). Gate CM-GITIGNORE-PRECOMMIT-AUDIT + paired mutation. Recreatable-content test: if a documented mechanism regenerates the file from sources, it's a build derivative and MUST be ignored.

Secret-leak intersection (§11.4.10 / CONST-042 / §12.1): a .env leak is BOTH a §11.4.30 and a §11.4.10 violation; rotation + post-mortem required.

Classification: universal (§11.4.17). Severity-equivalent to §11.4 PASS-bluff at the repository-hygiene layer. Composes with §1, §2, §9.1, §11.4.10, §11.4.12, §11.4.17, §11.4.18, §11.4.20, §11.4.25, §11.4.26, §11.4.27, §11.4.28, §11.4.29, CONST-047. See Constitution §11.4.30 for the full mandate.

Lowercase-snake_case-naming (§11.4.29, User mandate 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

"naming convention for Submodules and directories (applied deep into hierarchy recursively) - all directories and Submodules MSUT HAVE lowercase names with space separator between the words of ' ' character (snake-case)! All existing Submodules and directories which are not following this rule MUST BE renamed! ... NOTE: Rules lowercase / snake-case do apply to all project files as well and references to it and from them!"

Every directory / submodule / file MUST use lowercase snake_case. Non-compliant names MUST be renamed; every reference (configs, docs, source imports, governance files) updated atomically (reference drift = §11.4.29 violation of equal severity). Exceptions (common-sense, technology-preserving): language-mandated case for Java/Kotlin/Android/Apple/C#/Swift inside the language root; vendor third-party submodules; build artefacts.

Upstreams/ → upstreams/ transition: constitution submodule's install_upstreams.sh MUST support BOTH directory names during migration; lowercase wins when both present.

Project-Toolkit Upstreamable machinery fetched+pulled before any rename batch + itself complies. Lacking BOTH-dir support is a release blocker.

Test coverage of renames: regression test for reference resolution + full CONST-050(B) test-type matrix + anti-bluff wire-evidence.

Phased execution: brainstorming → phase plan → fine-grained tasks/subtasks → every change covered. §11.4.20 subagent delegation for cross-cutting rename sweeps.

Classification: universal (§11.4.17). No escape hatch beyond common-sense exceptions. Severity-equivalent to §11.4 PASS-bluff at the reference-integrity layer. Composes with §1, §11.4.12, §11.4.17, §11.4.18, §11.4.20, §11.4.25, §11.4.26, §11.4.27, §11.4.28, CONST-047. See Constitution §11.4.29 for the full mandate.

Submodules-as-equal-codebase + decoupling + dependency-layout (§11.4.28, User mandate 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

"All existing Submodules in the project that we are controlling and belong to some our organizations (vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Fact-ory — we can ALWAYS check dynamically using GitHub and GitLab CLIs) are equal parts of the project's codebase! We MUST work on that code as much as we do with main project's codebase! ... We MUST NEVER modify Submodules to bring into them any project specific context ... All Submodule dependencies that are used by Submodule MUST BE accessed from the root of the project! We MUST NOT have nested Submodule dependencies."

Three invariants. **(A) Equal-codebase:** every owned-by-us submodule (orgs: vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory — discoverable via gh/ghlab) is an equal part of the consuming project's codebase. Same engineering attention: analysis, extension, tests, gap-fill, bug-fix, documentation. Coverage ledgers list each submodule as in-scope. **(B) Decoupling:** NEVER inject project-specific context INTO submodules; they remain project-not-aware, reusable, modular, testable. When a submodule needs parent info, use configuration injection. **(C) Dependency-layout:** every dependency consumed by an owned submodule lives at <root>/<name>/ or <root>/submodules/<name>/ of the parent project. Nested own-org submodule chains FORBIDDEN — add the dependency at the parent root; the submodule reaches it via documented import/SDK/runtime resolver. Third-party submodules exempt.

Gates: CM-OWNED-SUBMODULE-EQUAL-ENGINEERING, CM-OWNED-SUBMODULE-DECOUPLING, CM-OWNED-SUBMODULE-LAYOUT. Paired mutations for each. Classification: universal (§11.4.17). No escape hatch. Severity-equivalent to a §11.4 PASS-bluff at the codebase-completeness

layer. Composes with §1, §3, §11.4.17, §11.4.20, §11.4.25, §11.4.26, §11.4.27, CONST-047. See Constitution §11.4.28 for the full mandate.

No-fakes-beyond-unit-tests + 100%-test-type-coverage (§11.4.27, User mandate 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

"Mocks, stubs, placeholders, TODOs or FIXMEs are allowed to exist ONLY in Unit tests! All other test types MUST interact with real fully implemented System! No fakes, empty implementations or bluffing is allowed of any kind! All codebase of the project MUST BE 100% covered with every supported test type."

Two invariants. **(A)** Mocks/stubs/fakes/placeholders/TODOs/FIXMEs/"for now" patterns PERMITTED only in unit-test sources; non-unit tests (integration, E2E, full-automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges, HelixQA) MUST exercise the real, fully implemented system. Production code MUST NOT import mock paths. Gate CM-NO-FAKES-BEYOND-UNIT-TESTS + paired mutation. **(B)** Codebase MUST be covered by every supported test type the domain warrants: unit, integration, E2E, full-automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges (vasic-digital/Challenges submodule fully incorporated), HelixQA (HelixDevelopment/HelixQA submodule fully incorporated, with full autonomous QA sessions executing every registered test bank).

Required dependency submodules (recursive per CONST-047): Challenges ([git@github.com:vasic-digital/Challenges.git](https://github.com/vasic-digital/Challenges.git)) + HelixQA ([git@github.com:HelixDevelopment/HelixQA.git](https://github.com/vasic-digital/HelixDevelopment/HelixQA.git)) + any other functionality submodules under vasic-digital/HelixDevelopment ors the project depends on. Pointers bumped to upstream HEAD in same commit as cascade work (§11.4.26 step 7).

Classification: universal (§11.4.17). Severity-equivalent to a §11.4 PASS-bluff at the release-gate layer. No escape hatch. Composes with §1, §7.1, §11.4.1–§11.4.26 (esp. §11.4.25 — strict per-type-of-test expansion). See Constitution §11.4.27 for full mandate.

Type-aware closure-status vocabulary (§11.4.33, User mandate 2026-05-15)

Every project that tracks work items by Type per §11.4.16 MUST close them with the Type-appropriate closure-status word: Bug → Fixed (→ Fixed.md), Feature → Implemented (→ Fixed.md), Task → Completed (→ Fixed.md). The (→ Fixed.md) suffix is preserved across all three so existing migration tooling (atomic Issues.md →

Fixed.md move per §11.4.19) keeps working. Generators treat the three terminal values as semantically equivalent (all closed, positive evidence captured) but preserve the literal in emitted docs. Closing a Feature with Fixed (→ Fixed.md) or a Task with Implemented (→ Fixed.md) is a §11.4.33 violation. Pre-build gate CM-CLOSURE-VOCAB-TYPE-AWARE. Composes with §11.4.15 / §11.4.16 / §11.4.19 / §11.4.23. Classification: universal (per §11.4.17). No escape hatch.

Reopened-source attribution (§11.4.34, User mandate 2026-05-15)

Every Issues.md heading whose ****Status:**** is Reopened **MUST** carry a ****Reopened-Details:**** line within 8 non-blank lines of the heading, capturing four sub-facts: **By:** AI or User; **On:** ISO date; **Reason:** one of { test-failed | manual-testing-detected | captured-evidence-contradicts | end-user-report | cycle-re-discovered | design-reconsidered } or explicit free text; **Evidence:** path or short description of the captured artefact. Reopens without evidence are §11.4.6 / §11.4.7 violations: the reopen IS a demotion-from-Fixed change. Issues_Summary.md Status column **MUST** distinguish Reopened sub-states by source (e.g., Reopened (AI: test-failed) vs Reopened (User: manual-testing)). Pre-build gate CM-ITEM-REOPENED-DETAILS mirrors §11.4.21 walk pattern. Composes with §11.4.6 / §11.4.7 / §11.4.15 / §11.4.21. Classification: universal (per §11.4.17). No escape hatch.

Canonical-root inheritance clarity (§11.4.35, User mandate 2026-05-15)

The constitution submodule's three files (constitution/Constitution.md, constitution/CLAUDE.md, constitution/AGENTS.md) ARE the canonical root — also called the parent files. Universal rules per §11.4.17 live here.

The consuming project's repository-root files (<project-root>/CLAUDE.md, <project-root>/AGENTS.md, optionally <project-root>/Constitution.md or equivalent) are consumer extensions. They open with the inheritance pointer (either @constitution/CLAUDE.md import or ## INHERITED FROM constitution/CLAUDE.md heading). Project-specific rules per §11.4.17 live here.

When in doubt: universal rule → constitution submodule; project-specific rule → consumer's file. Default consumer-side when uncertain. "Parent CLAUDE.md" / "root Constitution" → constitution submodule file at constitution/<filename>. "Project CLAUDE.md" / "this project's AGENTS.md" → consumer file at <project-root>/<filename>. Moving a rule between layers **MUST** be a visible commit — git mv + explicit "Lifted from to constitution per

§11.4.35" / "Demoted from constitution to per §11.4.35" line in the message. Pre-build gate CM-CANONICAL-ROOT-CLARITY. Composes with §11.4.17. Classification: universal (per §11.4.17). No escape hatch.

Mandatory install_upstreams on clone/add (§11.4.36, User mandate 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

"Every Submodule or Git repository we add or clone MUST BE upstreams installed using Upstreamable utility which MUST BE available through exported paths of the host system (in .bashrc or .zhrc) using install_upstreams command executed from the root of the cloned (added) repository - only if in it is Upstreams or upstreams directory present with bash script files (recipes) for all repository's upstreams!"

Every clone / add of a Git repository under any consuming project MUST be followed by install_upstreams invocation from the repository's root IF its tree contains upstreams/ (or legacy Upstreams/ per §11.4.29 transition) populated with *.sh recipe files. The utility (installed on operator's PATH via .bashrc/ .zshrc, implementation lives in this constitution submodule) reads recipe files + configures every declared upstream as a named git remote + fans out origin push URLs.

Skipping the invocation when upstreams/ is present silently breaks §2.1 (multi-upstream push is the norm). Gate CM-INSTALL-UPSTREAMS-ON-CLONE + paired mutation. Automation: incorporate-submodule (§11.4.31) auto-invokes; manual invocation also supported. Pre-commit check: git remote -v | grep -c push reports expected upstream count.

Classification: universal (§11.4.17). Composes with §2, §2.1, §3, §9.2, §11.4.17, §11.4.20, §11.4.28, §11.4.29, §11.4.30, §11.4.31. See Constitution §11.4.36 for the full mandate.

Fetch-before-edit (§11.4.37, User mandate 2026-05-15)

Forensic anchor — verbatim user mandate (2026-05-15):

"Make sure that feedback_fetch_before_edit memory rule is part of our constitution Submodule - the root Consitution, AGENTS.MD and CLAUDE.MD. Validate and verify that Proejct-Toolkit and all Submodules do inherit all of them!"

FIRST git-touching action of any session: git fetch --all --prune && git log --oneline HEAD..@{u} && git submodule foreach --recursive 'git fetch --all --prune --quiet'. If HEAD..@{u} is non-

empty, integrate (ff-merge / rebase / surface per §11.4.4) BEFORE any local edit, scanner, or test. Skipping on "nothing could have changed" is a §11.4.6 (no-guessing) violation — remote state is not knowable without a fetch.

The originating 2026-05-15 incident: agent ran `git remote remove gitee` as a fresh task, but the work had been committed upstream by a parallel agent 25 minutes earlier — local config edit was a no-op echo of already-merged history. A 30-second fetch would have caught it.

Scope: consuming project root + every owned submodule recursively (§11.4.28) + the constitution submodule itself (§11.4.26 step 1) + any dependency cloned via `incorporate-submodule` (§11.4.31) or `git submodule add` (§11.4.36). Gate CM-FETCH-BEFORE-EDIT-AUDIT + paired mutation per §1.1. Classification: universal (per §11.4.17). No escape hatch.

Installable-asset evidence (§11.4.38, User mandate 2026-05-17)

For any user-distributable build artifact (package, bundle, installer, or container image), tests/challenges MUST open the artifact and verify each user-visible asset is **present** and **non-degenerate**. A PASS without artifact-opening verification is a §11.4 PASS-bluff. The failure mode: source file exists → build packages it → source-layer checks pass → artifact produced with asset stripped or misconfigured, and no gate opens the artifact to verify.

Required per consuming project: one challenge script per artifact type that opens the produced artifact and verifies every declared user-visible asset. The challenge MUST run as part of the standard QA gate. Classification: universal (per §11.4.17). No escape hatch. See Constitution §11.4.38 for the full mandate.

Full-suite retest before release tag (§11.4.40, User mandate 2026-05-17)

Release tag MUST NOT be created until a COMPLETE retest with ALL existing tests runs on a clean baseline AFTER every workable item in the batch is done, fixed, polished, individually verified. Spot-check retests FORBIDDEN — miss interaction defects. Comprises 7 steps: (1) pre-build full sweep, (2) post-build full sweep, (3) on-device 4-phase cycle on EVERY owned device, (4) meta-test full mutation sweep, (5) Challenge bank full sweep, (6) Issues.md/Fixed.md state audit, (7) CONTINUATION.md sync check. Typically 12–48h elapsed, NOT optional, NOT abbreviated. Composes with §11.4.4 (per-fix retest still required at fix granularity) + §11.4.7 (full-suite is authoritative baseline for closures) + §11.4.39

(per-feature on-device validation runs as step 3). Classification: universal (per §11.4.17). No escape hatch. See Constitution §11.4.40 for the full mandate.

Pre-force-push merge-first (§11.4.41, User mandate 2026-05-17)

Forensic anchor — verbatim user mandate (2026-05-17):

"make sure we bring everything from branches to our side before forc push is done! Afer everything is safely and fully merged and all potential conflicts (if any) resolved, then do force push! make sure nothing isnlost, broken or corrupted on bith sides!"

Any force-push authorised under CONST-043 MUST be preceded by a mechanical 4-step merge-first pipeline: (1) fetch every remote (`git fetch --all --prune --tags`); (2) integrate every divergent commit locally (rebase / merge / operator-confirmed cherry-pick per appropriate strategy); (3) audit the integrated tree (no conflict markers anywhere in governance + source + test files; no file silently dropped; previously-passing tests still pass; captured-evidence artefacts still validate); (4) only THEN execute `git push --force-with-lease`.

Two-gate composition with CONST-043. §11.4.41 does NOT relax CONST-043 — it adds a SECOND mechanical gate. CONST-043 alone authorises a push that loses remote work; §11.4.41 alone risks pushing without operator awareness. Both required.

Three failure modes prevented: remote-side content loss (parallel sessions overwritten), stale-state acts (`--force-with-lease` reads stale local refs without prior fetch), conflict-driven corruption (markers committed verbatim — observed 2026-05-17 in `helix_qa` + containers governance files). Verification artefact: `docs/changelogs/<tag>.md` "Force-push merge-first audit" section captures fetch output, divergence log, integration strategy, conflict-marker scan, test delta, push output with lease SHA, + CONST-043 authorisation quote. Gate CM-FORCE-PUSH-MERGE-FIRST

- paired mutation. Classification: universal (§11.4.17). No escape hatch. Composes with §9.2, §11.4.4, §11.4.6, §11.4.26, §11.4.32, §11.4.37, §11.4.40, CONST-043, CONST-047. See Constitution §11.4.41 for the full mandate.

Iteration-discipline mandate (§11.4.42, User mandate 2026-05-18)

Work proceeds in priority-ordered cycles. Five mandatory steps per cycle: (1) select TOP + MIDDLE critical only (defer LOW until critical batch closed), (2) batch implementation per §11.4.4 +

§11.4.9, (3) smoke gate (<30 min), (4) ONLY if smoke GREEN and no new user/operator report → §11.4.40 full retest (12–48 h), (5) release-ready OR loop back. Composes with §11.4.4 / §11.4.7 / §11.4.9 / §11.4.34 / §11.4.40 — §11.4.42 is the meta-loop conductor binding them. Every PASS in smoke and full retest MUST carry positive captured evidence (§11.4.2 + §11.4.5). Tests AND Challenges bound equally. No escape hatch. Classification: universal (per §11.4.17). See Constitution §11.4.42 for the full mandate.

TDD-Fix-Discipline (§11.4.43, User mandate 2026-05-18)

Every fix follows 5-step workflow: **RED** (failing test FIRST, real product defect, captured evidence) → **LIVE-ADB-PROBE** (try on running device when feasible; INFEASIBLE for kernel / framework / HAL / vendor / init.rc / sepolicy / ro.* / Android.bp — must rebuild; cite LIVE_PROBE_INFEASIBLE: <reason> in commit) → **GREEN** (source patch, batched per §11.4.9, four-layer coverage per §11.4.4) → **VERIFY** (re-run RED test, PASSes under SAME conditions per §11.4.7, 10-iteration reliability per §11.4.42) → **DOCUMENT** (Issues.md → Fixed.md with type-aware closure §11.4.33, CLAUDE.md Applied Fixes row, changelog, guides, HelixQA bank, CONTINUATION.md §12.10 — all in SAME commit). Test-after-fix is a §11.4 PASS-bluff: test agrees with fix but does not catch the bug. No escape hatch. Classification: universal (§11.4.17). See Constitution §11.4.43 for the full mandate.

Document revision header (§11.4.44, User mandate 2026-05-18)

Every IN-scope tracked Markdown document (Issues.md / Issues_Summary.md / Fixed.md / Fixed_Summary.md / CONTINUATION.md / docs/guides/** / docs/research/** / docs/scripts/** / docs/changelogs/** / docs/superpowers/plans/** / docs/hardware/** / every other docs/*.md) carries **Revision:** N (monotonic positive integer)

- **Last modified:** YYYY-MM-DDTHH:MM:SSZ directly below the H1 title. CLAUDE.md / AGENTS.md / README / LICENSE / VERSION / rendered HTML / rendered PDF are OUT of scope. Agents MUST read the **Revision:** N literal from the document head when reporting document state, NEVER infer freshness from filename mtime or commit log — both can lie. Agents writing to an IN-scope doc MUST invoke scripts/doc_revision_bump.sh <file> as the final step before staging, OR rely on the pre-commit hook. Agents MUST NOT manually edit the revision number — only the bump script is authoritative.

Pre-build gates CM-DOC-REVISION-HEADER-PRESENT + CM-COVENANT-114-44-PROPAGATION + paired mutations. Composes with §12.10 (CONTINUATION.md header reuse), §11.4.12, §11.4.22, §11.4.23, §11.4.18. No escape hatch. Classification: universal (§11.4.17). See Constitution §11.4.44 for the full mandate.

Integration-status-doc maintenance (§11.4.45, User mandate 2026-05-18)

Every non-trivial domain integration MUST have a docs/<domain>/<integration>/Status.md carrying the §11.4.44 revision header + auto-synced HTML+PDF + auto-colored per §11.4.23 + sync wrapper + captured-evidence table (every claim cites the test log / recording / sink-probe path) + closed status vocabulary (PASS / FAIL / SKIP / PENDING_FORENSICS / OPERATOR-BLOCKED) + operator-blocked items at the top + reference from CONTINUATION.md §3 when non-terminal. §11.4.45 is the generic form of §12.10 applied to every integration domain.

Pre-build gates CM-COVENANT-114-45-PROPAGATION + CM-AF-INTEGRATION-STATUS-DOCS + 4 paired mutations. Composes with §11.4.5 / §11.4.12 / §11.4.13 / §11.4.15 / §11.4.22 / §11.4.23 / §11.4.44 / §12.10. No escape hatch. Classification: universal (§11.4.17). See Constitution §11.4.45 for the full mandate.

Validate-recent-work-before-post-flash-tests (§11.4.46, User mandate 2026-05-18)

Every post-flash run MUST first run a recent-work validation pass (targeted tests for Issues.md In-progress / Ready-for-testing / Re-opened + Fixed.md last-7-days + CONTINUATION.md §3 active work). Full test_all_fixes.sh runs ONLY after 100% green.

Helper: scripts/testing/recent_work_validate.sh --device <serial> writes /data/local/tmp/.recent_work_validated; full suite refuses without it. Each recent fix needs paired §11.4.43 RED-then-GREEN — naked GREEN is a bluff. Composes with §11.4.4 + §11.4.7 + §11.4.40

- §11.4.42 + §11.4.43 + §11.4.44 + §12.10. Pre-build gates CM-COVENANT-114-46-PROPAGATION + CM-AF-RECENT-WORK-VALIDATION-GATE + CM-AF-VALIDATION-ARTIFACT-FILE + paired mutations. Classification: universal (§11.4.17). No escape hatch. See Constitution §11.4.46 for the full mandate.

Firestore data review (§11.4.47, User mandate 2026-05-18)

Before every "bigger working round" (pre-build / pre-flash / pre-tag blocking; daily / post-deployment burn-in non-blocking) the operator/loop MUST execute `scripts/firebase/review_round.sh`. The pass queries Crashlytics (fatals + non-fatals + ANRs) + Analytics + Performance, classifies findings by severity, dedup- maps to existing Issues.md entries via a three-tier algorithm (exact Firestore Issue-ID match → stacktrace-similarity cluster hash → operator merge review), and drafts new Issue entries for unrecognised findings with full Firestore Console URL refs + §11.4.4(a) systematic-debugging output.

Five mandatory elements: 5-trigger cadence, 3-source query (all three sources), Issues.md output with Firestore metadata (Issue IDs + URL + Cluster Hash / KPI / Funnel), 3-tier dedup, and comprehensive root-cause analysis per Issue.

Pre-build gates CM-COVENANT-114-47-PROPAGATION + CM-AF-FIREBASE-REVIEW-CADENCE + CM-AF-FIREBASE-ISSUE-XREF + 3 paired mutations. Composes with §11.4.4 / §11.4.4(a) / §11.4.6 / §11.4.7 / §11.4.10 / §11.4.12 / §11.4.14 / §11.4.15 / §11.4.16 / §11.4.34 / §11.4.42 / §11.4.43 / §11.4.44 / §11.4.45 / §11.4.46. No escape hatch — no `--skip-firebase-review`. Classification: universal (§11.4.17). See Constitution §11.4.47 for the full mandate.

UI-driven video testing (§11.4.48, User mandate 2026-05-18)

Every test that asserts video playback on a secondary display MUST traverse the user-equivalent UI path (launcher icon → app home → content list → tile tap → playback → in-app pause/resume → back button stop). NOT Intent/Broadcast shortcuts (`am start -a VIEW, cmd media_session play`). Real `uiautomator` dump element resolution

- `input tap X Y`. Coverage: every video-capable app in `PRODUCT_PACKAGES` + every stream type (progressive HTTP / HLS / DASH / RTMP / file-local / DRM) + every codec (H.264/265/VP9/AV1/MPEG-2/MP4V video; AC-3/E-AC-3/TrueHD/DTS/DTS-HD/MLP/Opus/AAC audio). Secondary display verified via `ffprobe-on-captured-mp4` + VOM `activeDecoder` state. Arvus codec-state cross-check per §11.4.13 + §CG screenshot.

Per §11.4.4 four-layer: pre-build gate CM-AF-UI-DRIVEN-VIDEO-COVERAGE + propagation gate CM-COVENANT-114-48-PROPAGATION + on-device test framework at `device/rockchip/rk3588/tests/ui_driven/` (Layer 1 helper + Layer 2 per-app drivers + Layer 3 scenarios) + Layer 4 orchestrator `scripts/testing/run_ui_driven_video_suite.sh`

+ paired meta-test mutations. No escape hatch — no `--use-intent-shortcut`, `--skip-ui-traverse`, `--legacy-intent-mode` flag. Classification: universal (§11.4.17). See Constitution §11.4.48 for the full mandate.

Dual-approach testing (§11.4.49, User mandate 2026-05-18)

Every feature test exercising a user-visible behaviour MUST ship in TWO variants: a UI-driven variant (uiautomator-based, §11.4.48 surfaces A-E) AND an Intent/Broadcast-driven variant (`am start --es / am broadcast-based`). Either alone is a §11.4 PASS-bluff for the OPPOSITE half of the stack — UI catches app-side bugs; Intent catches framework/system-server bugs (MediaCodec.configure hook, IVideoOutputManager bind timing, broadcast permission gating).

Shared assertion base tests/lib/dual_approach_test_base.sh — both variants share `dat_init / dat_start_capture / dat_assert_codec_state / dat_assert_video_frames / dat_assert_audio_channels / dat_arvus_dashboard_capture / dat_cleanup / dat_report_finding`. Both variants gather identical evidence into mirror dirs `qa-results/dual_approach/ <F>/<run-ts>/ {ui,intent}/` so orchestrator diffs results and pinpoints which half of the stack contains a bug. Status mismatch between variants is itself a finding.

Kinopoisk 5.1 EAC3 is the canonical first implementation. Both variants are RED per §11.4.43 until the §CN decoder pipeline fix lands. Pre-build gates: `CM-COVENANT-114-49-PROPAGATION + CM-AF-DUAL-APPROACH-COVERAGE + CM-AF-KINOPOISK-5-1-DUAL-COVERAGE`. Three paired meta-test mutations. No escape hatch — no `--ui-only / --intent-only / --skip-dual` flag. Classification: universal (§11.4.17). See Constitution §11.4.49 for the full mandate.

Deterministic consistency (§11.4.50, User mandate 2026-05-18)

Every test that PASSES MUST have been executed N times (default N=3 normal tests, N=10 cycle-validation suites) against the same firmware MD5 + same device + same topology and produced IDENTICAL PASS in every iteration. A divergent N-iter run is auto-FAIL — there is no "first PASSED therefore X was a flake" path. §11.4.7's intermittent / transient / flake / flaky vocabulary is enforced MECHANICALLY by this mandate, not merely textually.

Coverage: every public API path (Activity / Service / Broadcast receiver / ContentProvider URI / IPC interface / JNI entry / sysprop write / sysfs node / init.rc trigger) MUST have ≥ 1 dedicated test that drives it. Untested paths surface in a feature-coverage-matrix audit; the threshold ratchets $70 \rightarrow 85 \rightarrow 95 \rightarrow 99$ over phases.

Reliability mechanism: `ab_run_n_times` helper in the project anti-bluff library loops, captures evidence-hash per iter, asserts all N hashes + exit codes identical. NO operator-facing escape converts divergence to PASS.

Pre-build gates: `CM-COVENANT-114-50-PROPAGATION` + `CM-AF-RELIABILITY-CHECK-WIRED` + `CM-AF-FEATURE-COVERAGE-MATRIX`. Three paired meta-test mutations. No escape hatch — no `--allow-flake`, `--first-pass-suffices`, `--skip-n-iter`, `--skip-coverage-audit` flag. Classification: universal (§11.4.17). See Constitution §11.4.50 for the full mandate.

Live-ADB-First maximization (§11.4.51, User mandate 2026-05-18)

Every fix MUST be classified by rebuild-requirement before commit using the project's per-file-class decision matrix. If `LIVE_ADB_TESTABLE` (on-device test scripts, host scripts, *atmosphere.sh boot scripts, persist.* properties, markdown docs, test fixture assets, HelixQA YAML banks), the operator MUST first apply the fix to the running device via `adb push / setprop / pm install -r / mount -o remount,rw`, run the §11.4.43 RED test live, capture PASS, THEN commit + rebuild + reflash. Commit footer: `LIVE_ADB_VALIDATED: yes`. If `REQUIRES_REBUILD` (kernel, framework Java/AIDL, native C++ in APEX, sepolicy, init.rc, `ro.*` properties, XML overlays, codec XML in APEX, `Android.bp/.mk`), the operator proceeds directly to source-side + rebuild. Commit footer: `REQUIRES_REBUILD: <reason>`. Mixed batches use partial.

§11.4.51 REFINES §11.4.43 step 2 with mechanical enforcement. Helper: `classify_fix_rebuild_requirement.sh` walks `git diff --name-only`, looks up each file against the matrix, emits per-file classification + recommended commit-message footer. Unmatched paths classify as `REQUIRES_REBUILD: unmatched-path` (safe default per §11.4.6).

Pre-build gates: `CM-COVENANT-114-51-PROPAGATION` + `CM-AF-CLASSIFY-FIX-HELPER-EXISTS` + `CM-AF-LIVE-ADB-FIRST-COMMIT-MARKER`. Three paired meta-test mutations. No escape hatch — no `--skip-classify` / `--assume-rebuild` / `--no-footer-required` flag.

Classification: universal (§11.4.17). See Constitution §11.4.51 for the full mandate.

Autonomous-Validation (§11.4.52, User mandate 2026-05-18)

Forensic anchor — verbatim user mandate:

"Make sure we have full automation tests which will do all this work in full automation! IMPORTANT: Make sure that all existing tests and Challenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected! execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"

Every user-facing feature MUST have at least one autonomous validation path: end-to-end via adb shell + scripted automation, captured runtime evidence per §11.4.5, PASS/FAIL verdict WITHOUT human presence. Operator-attended tests are SUPPLEMENTARY, never PRIMARY. A feature whose ONLY path is operator-attended is a §11.4.52 violation: the path does not scale to CI, does not run on every commit, does not survive operator unavailability, and produces the exact "tests pass but feature doesn't work for users" failure mode §11.4 forbids.

Acceptable autonomous paths: instrumentation APK (SDK-API exercises + JSON result file), headless intent dispatch + state poll (am start --es + dumpsys//proc/<pid>/maps polling), ADB-driven ui-automator (ONLY if hierarchy has ≥1 clickable node — empty hierarchy demands fallback to APK/intent), network-side sink probe (§11.4.13), HelixQA autonomous QA session (§11.4.27).

Coverage ledger (§11.4.25) classifies each feature as AUTONOMOUS_VERIFIED / AUTONOMOUS_DESIGNED / OPERATOR_ATTENDED_ONLY / NOT_APPLICABLE. OPERATOR_ATTENDED_ONLY blocks release until migrated; cite a tracked work item per §11.4.15 + §11.4.16.

Pre-build gates CM-COVENANT-114-52-PROPAGATION + CM-AF-AUTONOMOUS-PATH-PER-FEATURE. Paired mutations. No escape hatch — no --allow-operator-attended-only / --skip-autonomous-path / --manual-validation-suffices flag.

Classification: universal (§11.4.17). See Constitution §11.4.52 for the full mandate.

Fixed_Summary parity (§11.4.53, User mandate 2026-05-18)

Forensic anchor — verbatim user mandate (2026-05-18T17:55Z):

"Note: Just like for Issues we have Issues_Summary, for Fixed we MUST HAVE Fixed_Summary - like all other docs: ALWAYS in sync and up to date and ALWAYS exported into the PDF and HTML!"

docs/Fixed_Summary.md is the symmetric short-form summary of docs/Fixed.md. MUST be regenerated whenever Fixed.md changes. HTML + PDF exports MUST travel with the markdown. Stale exports are §11.4.53 violations regardless of whether the underlying .md is correct. Same discipline as §11.4.12 Issues_Summary applied to Fixed.md.

Generator: scripts/testing/generate_fixed_summary.sh (canonical). Auto-sync wrapper: scripts/testing/sync_issues_docs.sh regenerates BOTH Issues_Summary AND Fixed_Summary in one shot, exports HTML + PDF, colorizes per §11.4.23, re-renders PDFs. MUST be invoked after any edit to Fixed.md. No --issues-only flag exists.

Sort order: closure date DESC (most-recent-Fixed first), §-letter / Fix-# secondary. Documented at top of generated file.

Composes with §11.4.12 (Issues_Summary sibling), §11.4.19 (atomic migration trigger), §11.4.23 (colorizer), §11.4.33 (type-aware closure terminal values), §11.4.44 (revision header), §12.10 (CONTINUATION.md resumption).

Pre-build gates CM-FIXED-SUMMARY-SYNC + CM-COVENANT-114-53-PROPAGATION. Paired mutations. No escape hatch — no --skip-fixed-summary-sync, --issues-only, --summary-not-applicable flag.

Classification: universal (§11.4.17). See Constitution §11.4.53 for the full mandate.

ATM-NNN ticket identifier (§11.4.54, User mandate 2026-05-19)

Forensic anchor — verbatim user mandate (2026-05-19):

"Every workable item in Issues / Issues_Summary / Fixed / Fixed_Summary MUST carry a stable, unique, auto-incremental ATM-NNN ticket identifier."

Every workable item heading in docs/Issues.md AND docs/Fixed.md MUST carry [ATM-NNN] (form ## \$X.Y. [ATM-NNN] <title>, zero-padded ≥3 digits). Allocated by scripts/testing/assign_atm_ticket_ids.sh and persisted to scripts/testing/.atm_ticket_state.json (jsonl). Identifiers are monotonic, never renumbered, never reused, no gaps. Issues_Summary.md and Fixed_Summary.md MUST carry an ATM ID column as the left-most data column.

Composes with §11.4.15 + §11.4.16 + §11.4.19 + §11.4.33 + §11.4.12 + §11.4.53 + §11.4.55 + §11.4.57.

Pre-build gates CM-ATM-TICKET-IDS-COMPLETE + CM-ATM-TICKET-IDS-UNIQUE + CM-ATM-TICKET-IDS-MONOTONIC + CM-COVENANT-114-54-PROPAGATION. Paired mutations. No escape hatch.

Classification: universal (§11.4.17). See Constitution §11.4.54 for the full mandate.

Reopens-history tracking + per-item Reopens.md (§11.4.55, User mandate 2026-05-19)

Forensic anchor — verbatim user mandate (2026-05-19):

"Add a Reopens-count column. For any item whose reopens-count > 0, create docs/issues/ATM-NNN/Reopens.md (+ HTML + PDF) with comprehensive reopen history + Fixed cycles."

Every workable item with `reopens_count > 0` MUST have docs/issues/<ATM-NNN>/Reopens.md (+ HTML + PDF). Required content: §11.4.44 revision header, item identification, cycle counters, chronological timeline (By AI/User per §11.4.34, On ISO date, Event, Reason, Evidence, Outcome), reasoning chain for each closure, most-recent state-change pointer.

Issues_Summary.md and Fixed_Summary.md MUST carry a Reopens column; count > 0 hyperlinks to per-item Reopens.md.

Composes with §11.4.34 (per-event capture), §11.4.54 (ATM-NNN path), §11.4.44 (revision header), §11.4.45 (Status.md analogue), §11.4.53 (Fixed_Summary parity — Reopens column symmetric).

Pre-build gates CM-REOPENS-DOC-EXISTS-WHEN-COUNT-GT-ZERO + CM-REOPENS-DOC-REVISION-HEADER + CM-REOPENS-COL-IN-SUMMARIES + CM-COVENANT-114-55-PROPAGATION. Paired mutations. No escape hatch.

Classification: universal (§11.4.17). See Constitution §11.4.55 for the full mandate.

Status_Summary parity + two-audience format (§11.4.56, User mandate 2026-05-19)

Forensic anchor — verbatim user mandate (2026-05-19):

"Every Status.md doc gets a Status_Summary parity companion ALWAYS in sync, exported to HTML + PDF. Two-page format: page 1 = non-developer audience (team-specific), page 2 = software engineer summary."

For every docs/<domain>/<integration>/Status.md a companion Status_Summary.md MUST exist with: §11.4.44 revision header; **Page 1 — For the** (audience-specific — audio team for docs/dolby/*, video team for docs/video/*, etc.) plain- language summary, What works, What's broken or pending, Operator actions — NO §-letter jargon, NO captured-evidence file paths; **Page 2 — For software engineers** — §-letter refs, gate names, commit hashes, captured-evidence paths, ATM-NNN cross-refs. HTML + PDF exports travel with the markdown.

Generator: scripts/testing/generate_status_summary.sh <Status.md path>. Invoked from sync_integration_status.sh (§11.4.45 wrapper) on every Status.md update.

Composes with §11.4.45 (Status.md — Status_Summary COMPLEMENTS), §11.4.12 + §11.4.53 (parity discipline), §11.4.44 (revision header), §11.4.23 (colorizer), §12.10 (CONTINUATION.md).

Pre-build gates CM-STATUS-SUMMARY-EXISTS-FOR-EVERY-STATUS + CM-STATUS-SUMMARY-TWO-AUDIENCE + CM-STATUS-SUMMARY-REVISION-HEADER + CM-COVENANT-114-56-PROPAGATION. Paired mutations. No escape hatch.

Classification: universal (§11.4.17). See Constitution §11.4.56 for the full mandate.

README.md doc-link section + revision metadata (§11.4.57, User mandate 2026-05-19)

Forensic anchor — verbatim user mandate (2026-05-19):

"Add a doc-link section to README.md — links to Issues + Issues_Summary + Fixed + Fixed_Summary + CONTINUATION + ALL Status docs + their exports. Each link shows revision + last-modified."

Every project's top-level README.md MUST contain a section titled Tracked-Items + Status Documents (heading MUST contain literal Tracked-Items). The section is a markdown table with columns: Document, Last modified, Revision, Markdown, HTML, PDF. Links to: Issues.md + Issues_Summary.md, Fixed.md + Fixed_Summary.md, CONTINUATION.md, every docs/**/Status.md + its Status_Summary.md pair. Status docs auto-discovered by the generator via `find docs -name 'Status.md'`.

Generator: scripts/testing/update_readme_doc_links.sh extracts each doc's revision + last-modified from its §11.4.44 header, renders the table, replaces the section between explicit markers (<!-- doc-link-section:begin --> / <!-- doc-link-section:end -->). Invoked from sync_issues_docs.sh AND sync_integration_status.sh.

Composes with §11.4.12 + §11.4.19 + §11.4.53 (Issues / Fixed / summaries), §11.4.44 (revision-header data source), §11.4.45 + §11.4.56 (Status pairs), §12.10 (CONTINUATION.md).

Pre-build gates CM-README-DOC-LINK-SECTION-PRESENT + CM-README-DOC-LINK-ROWS-COMPLETE + CM-README-DOC-LINK-FRESHNESS + CM-COVENANT-114-57-PROPAGATION. Paired mutations strip markers, remove rows, backdate Last modified, strip anchor literal. No escape hatch.

Classification: universal (§11.4.17). See Constitution §11.4.57 for the full mandate.

Parallel-development methodology (§11.4.58, User mandate 2026-05-19)

Forensic anchor — verbatim user mandate (2026-05-19T~05:00Z MSK):

"We MUST CREATE adjusted improved version of working methodology where multiple workable items could be done in parallel, all come into the central (main) branch as they are done, then we at particular moment rebuild and reflash the System and in background full testing with validation and verification is done. Each parallel work on one workable (or more) item(s) must use parallel agents as much as possible! ... Writing in depth tests (all supported types of the tests) with Challenges and full HelixQA use is MANDATORY! Every test we execute besides executed with success MUST RESULT in proof that actual functionality being tested REALLY DOES WORK with NO BLUFF of any kind!"

Project work proceeds through the **Parallel Work Unit (PWU) pipeline**. Each PWU is a self-contained workable item with: ATM-NNN identifier per §11.4.54, Issues.md entry per §11.4.15+§11.4.16, file-scope manifest, §11.4.43 RED test, source patch, pre-build gate, post-flash test, paired meta-test mutation per §1.1, HelixQA Challenge bank entry, captured-evidence directory per §11.4.5+§11.4.52.

5-stage pipeline: Stage 1 DEVELOP (parallel PWU agents in isolated worktrees) → Stage 2 MERGE (serial via `commit_all.sh` flock + §11.4.41 4-step merge-first) → Stage 3 REBUILD+FLASH (parallel where hardware allows) → Stage 4 VALIDATE (parallel D3+D4+meta-test+coverage) → Stage 5 SWEEP (parallel HelixQA + Fixed.md migration + README refresh). Stage 1 of round N+1 overlaps with Stages 4-5 of round N — throughput multiplier.

Synchronization: 4-layer lock hierarchy (parent flock / per-submodule git / contention-path advisory locks / per-PWU worktree). Disjoint-scope PWUs fully parallel.

Anti-bluff merge-time enforcement (mandatory, all four):

RED-test captured (§11.4.43), paired meta-test mutation (§1.1), 3-iter deterministic-consistency (§11.4.50), captured-evidence per §11.4.5. Metadata-only / configuration-only / absence-of-error PASS REJECTED. HelixQA Challenge MANDATORY for every user-visible PWU.

Pre-build gates CM-PWU-LOCK-HIERARCHY + CM-PWU-ANTI-BLUFF-COVERAGE

- CM-PWU-MERGE-QUEUE-DISCIPLINE + CM-PWU-PARALLEL-AGENT-LIMIT + CM-COVENANT-114-58-PROPAGATION. Paired mutations cover each gate. No escape hatch.

Classification: universal (§11.4.17). See Constitution §11.4.58 for the full mandate. Project-specific implementation reference in consumer-side docs/guides/PARALLEL_DEVELOPMENT_METHODODOLOGY.md.

README always-sync (§11.4.59, User mandate 2026-05-19)

README.md is a §11.4.12-class always-sync document. HTML + PDF exports refresh on every update via scripts/testing/sync_readme_export.sh (pandoc + weasyprint); auto-invoked by sync_issues_docs.sh so a single doc-sync run refreshes Issues / Issues_Summary / Fixed / Fixed_Summary / CONTINUATION / README (md + html + pdf). README carries a §11.4.44 revision header and a Documentation Map section linking to every Status.md + Status_Summary.md + spec + plan + guide + script-companion doc + changelog + the constitution submodule, plus per-audience navigation (end user / developer / QA / agent). Pre-build gate CM-README-EXPORT-SYNC enforces mtime parity (README.html + README.pdf ≥ README.md). Paired meta-test mutation backdates HTML+PDF → gate FAILs. No escape hatch — no --skip-readme-sync, --no-readme-export, --readme-stale-OK flag. Composes with §11.4.12

- §11.4.18 + §11.4.44 + §11.4.45 + §11.4.53 + §11.4.56 + §11.4.57 + §12.10.

Classification: universal (§11.4.17). See Constitution §11.4.59 for the full mandate.

Documentation always-sync composite covenant (§11.4.60, User mandate 2026-05-19)

Eight doc classes — Issues, Issues_Summary, Fixed, Fixed_Summary, CONTINUATION, README, every Status.md (domain-scoped), every Status_Summary.md — MUST be in sync across .md + .html + .pdf. Composite pre-build gate CM-DOCS-COMPOSITE-SYNC walks all 8 classes (Status fleet via recursive docs/** find), FAILs the build if ANY .html or .pdf mtime is older than

its .md. Per-class anchors §11.4.12 / §11.4.44 / §11.4.45 / §11.4.53 / §11.4.56 / §11.4.57 / §11.4.59 / §12.10 govern individually; the composite catches what they miss when one is silently bypassed. Paired mutation backdates docs/Issues.html → gate FAILs. No escape hatch.

Classification: universal (§11.4.17). See Constitution §11.4.60 for the full mandate.

Credentials-handling (§11.4.10)

Forensic anchor — verbatim user mandate (2026-05-12):

"Credentials or any secret and sensitive data MUST NOT leak!"

.env patterns git-ignored project-wide. Runtime-load-only — tests load credentials at execution time from operator-populated files under scripts/testing/secrets/ (or per-project equivalent). Test scripts MUST NEVER print or log credentials.

Host-session safety (§12)

Project procedures MUST NOT use more than **60%** of total system RAM. Heavy work wrapped in bounded execution scopes so the kernel OOM-kills only the scope — user@<uid>.service stays alive. Hibernating / suspending / logging out the user is FORBIDDEN (directly OR indirectly).

Continuation document (§12.10)

docs/CONTINUATION.md MUST exist at the project root and reflect the live state of the work. Every non-trivial state change updates it in the same commit. Any agent must be able to resume work exactly where the previous session left off by reading this single file.

Data safety (§9)

Hardlinked backup before any destructive operation. Post-op gate MUST pass. Force-push requires explicit per-session human authorization. Every history rewrite gets a docs/changelogs/ audit entry.

CLI workflow expectations

1. **Read before write.** Use file-reading tools to inspect the current state before editing. Edits in the dark are how bugs sneak in.

2. **Use the project's commit wrapper.** Direct `git add / git commit` is forbidden unless explicitly carved out.
3. **Commit messages cite forensic evidence** for any non-trivial fix (logs, captures, /sys readings, dropbox entries, strace). Speculative narratives are §11.4.6 violations.
4. **Update CONTINUATION.md** in the same commit as the work (§12.10).
5. **Run pre-build / post-build / runtime gates** before declaring a change complete. NEVER report "done" without running the relevant gate.
6. **Cite external research sources** for non-trivial fixes (§11.4.8). Either a citation OR the literal "NO external solution found — original work".
7. **Refuse to bluff.** If a step requires information you don't have, say so and STOP. Don't guess.

Hierarchy of project authority

When a project files declares conflicting rules with this base AGENTS.md, resolve in this order:

1. **Project's docs/guides/<PROJECT>_CONSTITUTION.md** — most specific, project-tailored.
2. **constitution/Constitution.md** (this submodule) — universal.
3. **Project's root CLAUDE.md / AGENTS.md** — agent-facing summaries; lossy compared to the Constitution.
4. **This constitution/CLAUDE.md / AGENTS.md** — universal agent-facing summary; lossy compared to the Constitution.

If a project's own files contradict the Constitution, that is itself a Constitution violation and MUST be reported. Do NOT silently follow the contradicting project file.

When stuck

- Read constitution/Constitution.md.
- Cross-reference the project's CLAUDE.md / AGENTS.md.
- If still unclear, ask the operator. Do NOT guess. Do NOT bluff.

§11.4.65 — Universal Markdown export mandate (User mandate, 2026-05-19)

Every Markdown document inside the project that is NOT part of an application or service's source-code tree MUST have synchronized .html and .pdf siblings. Includes: project-root *.md, docs/**/*.md, scripts/**/*.*.md (doc-format companion docs), owned-submodule top-level README.md / CLAUDE.md / AGENTS.md / CHANGELOG.md and their docs/**/*.*.md, constitution/**/*.*.md,

owned HelixQA submodules' equivalents. Excludes: external/**, prebuilts/**, packages/modules/**, kernel-5.10/**, out/**, build/**, application/service source-code trees, and third-party submodules NOT in the owned set. Every edit triggers regeneration via scripts/testing/sync_all_markdown_exports.sh (pandoc HTML + weasyprint PDF, timeout 60 per file, capped at 500 candidates). HTML + PDF mtime MUST be $\geq$ source .md mtime at all times.

Pre-build gates CM-UNIVERSAL-MARKDOWN-EXPORT-SYNC + CM-COVENANT-114-65-PROPAGATION. Paired meta-test mutations. Composes with §11.4.12 / §11.4.18 / §11.4.23 / §11.4.44 / §11.4.45 / §11.4.53 / §11.4.59 / §11.4.60 / §11.4.63 / §11.4.64. No escape hatch — no --skip-md-exports, --no-pdf-only, --md-export-not-applicable flag.

Canonical authority: constitution submodule Constitution.md §11.4.65.

Non-compliance is a release blocker regardless of context.